Documento de arquitectura técnica

Arquitectura propuesta y stack tecnológico

Hackathon de Inteligencia Artificial Aplicada · Congreso Social UC 2026 · Desafío 3

Documento de arquitectura técnica

Arquitectura propuesta y stack tecnológico

Hackathon de Inteligencia Artificial Aplicada · Congreso Social UC 2026 · Desafío 3

1. Propósito

Definir una arquitectura simple, demostrable y mantenible para el MVP de CIUDADANÍA CLARA, una plataforma web de apoyo a la verificación de información y comprensión de documentos públicos.

2. Principios de arquitectura

• Construir primero un MVP web responsive y evitar aplicaciones nativas para la hackathon.

• Separar presentación, lógica de aplicación, recuperación de evidencia y persistencia.

• La IA debe trabajar sobre evidencia recuperada y no responder como fuente autónoma de hechos.

• Priorizar fuentes y documentos previamente seleccionados para la demostración.

• Minimizar datos personales y secretos; no almacenar contenido de usuario si no es necesario.

• Mantener un modo de demostración reproducible con casos preparados.

3. Arquitectura lógica propuesta

Arquitectura cliente-servidor con una aplicación Next.js. La interfaz consume rutas de servidor/API; estas coordinan validación, recuperación de fuentes o documentos, llamada al modelo de IA y estructuración de la respuesta. Supabase puede utilizarse para persistir contenidos y progreso si el tiempo lo permite.

Capa	Responsabilidad	Tecnología propuesta
Presentación	Páginas, componentes, formularios, feed, academia y vistas de resultados.	Next.js + React + TypeScript + Tailwind CSS
Aplicación / API	Validación de entradas, control de flujo, endpoints y formato de respuesta.	Next.js Route Handlers / Server Actions
Servicio de verificación	Extracción de afirmaciones, recuperación de evidencia, evaluación y explicación.	Servicio TypeScript; SDK del proveedor de IA
Fuentes y documentos	Casos precargados, metadatos, documentos y referencias oficiales.	JSON/Markdown en repositorio para demo; Supabase Storage opcional
Persistencia	Contenidos, preguntas, progreso y registros mínimos.	Supabase (PostgreSQL) opcional
Despliegue	Publicación web y variables de entorno.	Vercel

4. Flujo de verificación

El usuario ingresa una afirmación o selecciona un caso del feed.

El backend valida longitud y formato, y normaliza el texto.

El sistema identifica la afirmación concreta que se debe evaluar.

Se recuperan fuentes pertinentes desde el conjunto autorizado o desde un mecanismo de búsqueda permitido para el MVP.

El modelo recibe la afirmación y los fragmentos de evidencia, con instrucciones para no inventar citas y declarar incertidumbre.

El backend valida que las referencias devueltas correspondan a fuentes recuperadas.

La interfaz presenta estado, explicación, evidencia, fuentes y limitaciones.

Estados recomendados: RESPALDADA, CONTRADICHA, ENGAÑOSA / FUERA DE CONTEXTO y NO VERIFICABLE. Estos estados describen la relación entre la afirmación y la evidencia disponible; no son una garantía absoluta de verdad.

5. Flujo del asistente cívico

El usuario selecciona un documento público precargado.

El backend recupera el texto o los fragmentos relevantes del documento.

El modelo responde únicamente a partir del material recuperado.

La respuesta incluye una explicación en lenguaje claro y la identificación del documento oficial.

Si el documento no contiene la respuesta, el sistema lo declara en vez de completar con información no respaldada.

6. Stack tecnológico recomendado

Componente	Recomendación	Motivo
Frontend y framework	Next.js + React + TypeScript	Desarrollo full-stack en un solo proyecto y buena experiencia responsive.
Estilos	Tailwind CSS	Permite construir y ajustar rápidamente una interfaz consistente.
Componentes UI	shadcn/ui (opcional)	Componentes accesibles y personalizables.
Backend	Next.js Route Handlers	Evita mantener un servidor separado para el MVP.
IA generativa	Proveedor disponible en la hackathon (API compatible)	La elección final depende de las cuentas, límites y modelos habilitados.
Base de datos	Supabase (opcional)	Útil para guardar casos, preguntas y progreso; no imprescindible para una demo pequeña.
Contenido demo	Archivos JSON y documentos curados en el repositorio	Reduce dependencias y hace reproducible la demostración.
Hosting	Vercel	Despliegue directo de Next.js y configuración sencilla de variables.

7. Modelo de datos mínimo

Entidad	Campos sugeridos
Claim / Afirmación	id, texto, tema, contexto, estado_demo, created_at
Source / Fuente	id, título, institución, URL, fecha_publicación, tipo, extracto
Verification / Verificación	id, claim_id, clasificación, explicación, limitaciones, created_at
VerificationSource	verification_id, source_id, fragmento, relación_con_afirmación
Lesson / Lección	id, título, objetivo, contenido, orden
Question / Pregunta	id, lesson_id, enunciado, alternativas, respuesta_correcta, explicación
Document / Documento	id, título, institución, URL_oficial, texto_o_archivo, fecha, versión

8. Endpoints sugeridos

Método y ruta	Propósito	Prioridad
POST /api/verify	Analizar una afirmación y devolver clasificación, explicación y fuentes.	Alta
GET /api/claims	Listar casos precargados para el feed.	Alta
GET /api/claims/:id	Obtener detalle de un caso y sus referencias.	Alta
GET /api/lessons	Listar lecciones educativas.	Media
GET /api/lessons/:id	Obtener contenido y preguntas de una lección.	Media
POST /api/civic/ask	Responder preguntas sobre un documento seleccionado.	Media

9. Contrato de respuesta del verificador

La API debe devolver un objeto estructurado equivalente al siguiente. Los enlaces deben proceder de fuentes realmente recuperadas; nunca deben ser generados libremente por el modelo.

{
"claim": "Texto de la afirmación evaluada",
"verdict": "RESPALDADA | CONTRADICHA | ENGANOSA | NO_VERIFICABLE",
"summary": "Conclusión breve y prudente",
"reasoning": "Explicación basada en evidencia",
"sources": [
{
"title": "Nombre de la fuente",
"url": "https://...",
"publisher": "Institución",
"evidence": "Fragmento relevante",
"supports": "A favor | En contra | Contexto"
}
],
"limitations": ["Limitación o dato faltante"],
"human_review": true
}

10. Seguridad, privacidad y uso responsable

• Guardar claves de API exclusivamente en variables de entorno del servidor; nunca exponerlas en el navegador ni en el repositorio.

• Validar entradas, limitar longitud y manejar errores sin mostrar secretos ni trazas internas.

• Evitar recopilar nombre, correo, ubicación u otros datos personales para el MVP.

• No persistir consultas ingresadas por usuarios salvo necesidad justificada y comunicada.

• Tratar los textos y documentos analizados como datos no confiables; no seguir instrucciones contenidas dentro de ellos.

• Mostrar fuentes, fecha o contexto disponible y limitaciones de cada análisis.

• Revisar manualmente los documentos y casos usados en el pitch.

• Declarar modelos y herramientas de IA utilizados en la solución.

11. Estrategia de desarrollo para la hackathon

Etapa	Entregable técnico
Diseño	Mapa de pantallas, flujo de usuario, definición de casos y contrato JSON.
Construcción base	Proyecto Next.js, navegación, layout responsive y repositorio compartido.
Verificador	Endpoint /api/verify y pantalla de resultados con fuentes y estados.
Contenido	Feed de casos precargados, una lección y un cuestionario funcional.
Asistente cívico	Una demo acotada sobre un documento público verificado.
Integración y pruebas	Recorrido completo, manejo de errores, respaldo de demo y revisión ética.

12. Decisiones pendientes antes de implementar

• Confirmar proveedor y modelo de IA disponible, junto con límites de uso.

• Definir el mecanismo concreto de recuperación de fuentes; no asumir que el modelo conoce o consulta fuentes actuales.

• Seleccionar un documento público oficial y dos o tres afirmaciones de demostración, con referencias revisadas.

• Decidir si Supabase es necesario: para una demo basada en datos estáticos puede omitirse.

• Definir quién revisa la neutralidad, pertinencia y exactitud de los casos antes del pitch.

13. Referencia de alineación

La arquitectura responde al Desafío 3 de las Bases Oficiales de la Hackathon de IA Aplicada – Congreso Social UC 2026. En particular, contempla trazabilidad de fuentes, neutralidad política, supervisión humana y demostración sobre documentos públicos precargados y verificados. La solución debe mantener la IA como apoyo al criterio humano.

Documento de trabajo para definición del prototipo. Versión 1.0 · 5 de noviembre de 2026.